

# Smart Lender: AI-Powered Loan Approval and Fraud Detection System

## Project Description:

Smart Lender is an AI-powered loan management system that streamlines the loan approval process by combining machine learning with automated document verification and fraud detection. The platform enables customers to apply for loans online, upload required documents, and receive faster, more accurate loan decisions while reducing manual effort for financial institutions.

The system includes a Loan Application Module, where users submit their personal, financial, and employment details, a Document Verification Module, which validates uploaded identity and income documents, a Fraud Detection Module, which uses machine learning algorithms to identify suspicious or fraudulent loan applications, and a Loan Eligibility Prediction Module, which evaluates an applicant's eligibility based on financial and historical data. An Admin Dashboard allows loan officers to review applications, monitor fraud alerts, and approve or reject loan requests efficiently.

Built using React.js for the frontend, Flask (Python) for the backend, MySQL for data storage, and Scikit-learn for machine learning, Smart Lender provides a secure, scalable, and user-friendly platform. The application follows a modular architecture with role-based authentication, secure data management, and REST APIs, making it suitable for modern digital lending solutions.

## Scenarios

### Scenario 1: Quick Loan Application

A customer registers on the Smart Lender platform, logs in, fills out the loan application form, uploads identity and income documents, and submits the application. The system verifies the information, performs fraud detection, predicts loan eligibility, and forwards the application for approval.

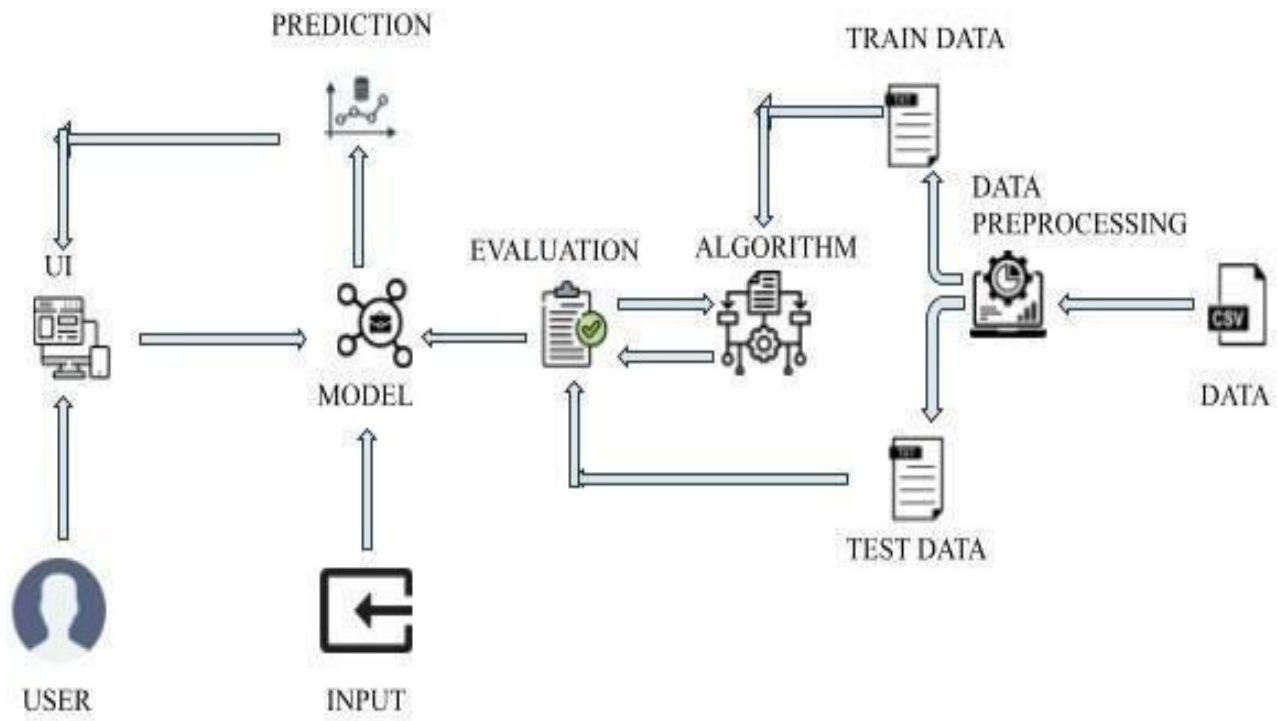
### Scenario 2: Fraudulent Loan Detection

A fraudster attempts to submit a loan application using fake or manipulated documents. The machine learning fraud detection module analyzes the submitted data, identifies suspicious patterns, assigns a high fraud risk score, and alerts the loan officer before any approval decision is made.

### Scenario 3: Loan Approval Process

A loan officer logs into the admin dashboard and reviews verified applications along with eligibility scores and fraud risk reports. Based on the system's recommendations, the officer approves eligible applications, rejects fraudulent ones, and the applicant receives an email or SMS notification with the final loan status.

### Technical Architecture:



**Description:** Smart Lender provides a secure and reliable loan management system by protecting user information and ensuring safe access through authentication and authorization. The system verifies user identities, stores data securely, and helps prevent unauthorized access to sensitive financial information. It also keeps users informed by sending notifications about loan application status, document verification, approval, rejection, and other important updates. This improves communication, increases transparency, and enhances the overall user experience.

## Pre-requisites:

**Python Documentation:** <https://www.python.org/doc/>

**Flask Framework:** <https://flask.palletsprojects.com/>

**Scikit-learn Documentation:** <https://scikit-learn.org/stable/documentation.html>

**Pandas Documentation:** <https://pandas.pydata.org/docs/>

**NumPy Documentation:** <https://numpy.org/doc/>

**Job lib Documentation:** <https://joblib.readthedocs.io/>

**HTML & CSS Tutorials (W3Schools):** <https://www.w3schools.com/>

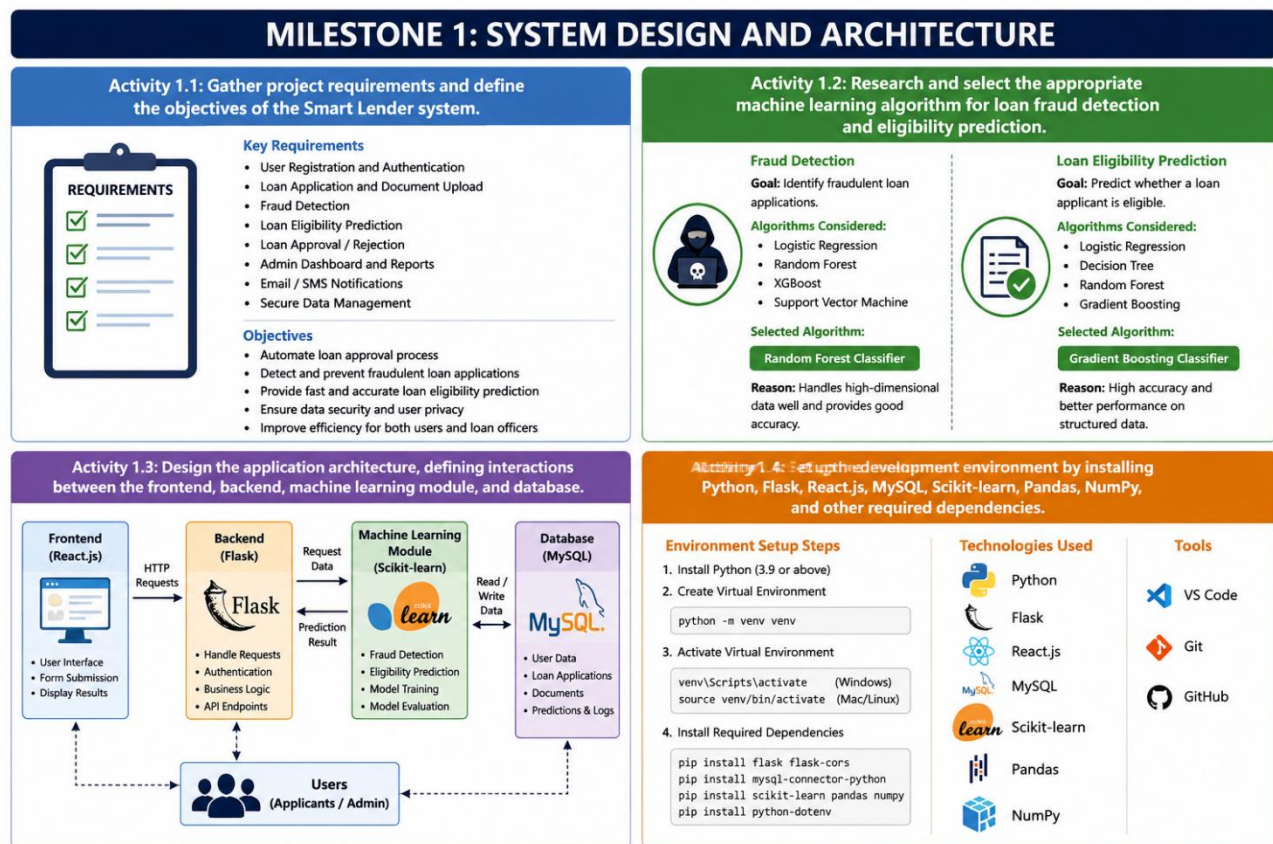
**Visual Studio Code:** <https://code.visualstudio.com/docs>

**Git Documentation:** <https://git-scm.com/docs>

## Project Workflow:

### Milestone 1: System Design and Architecture

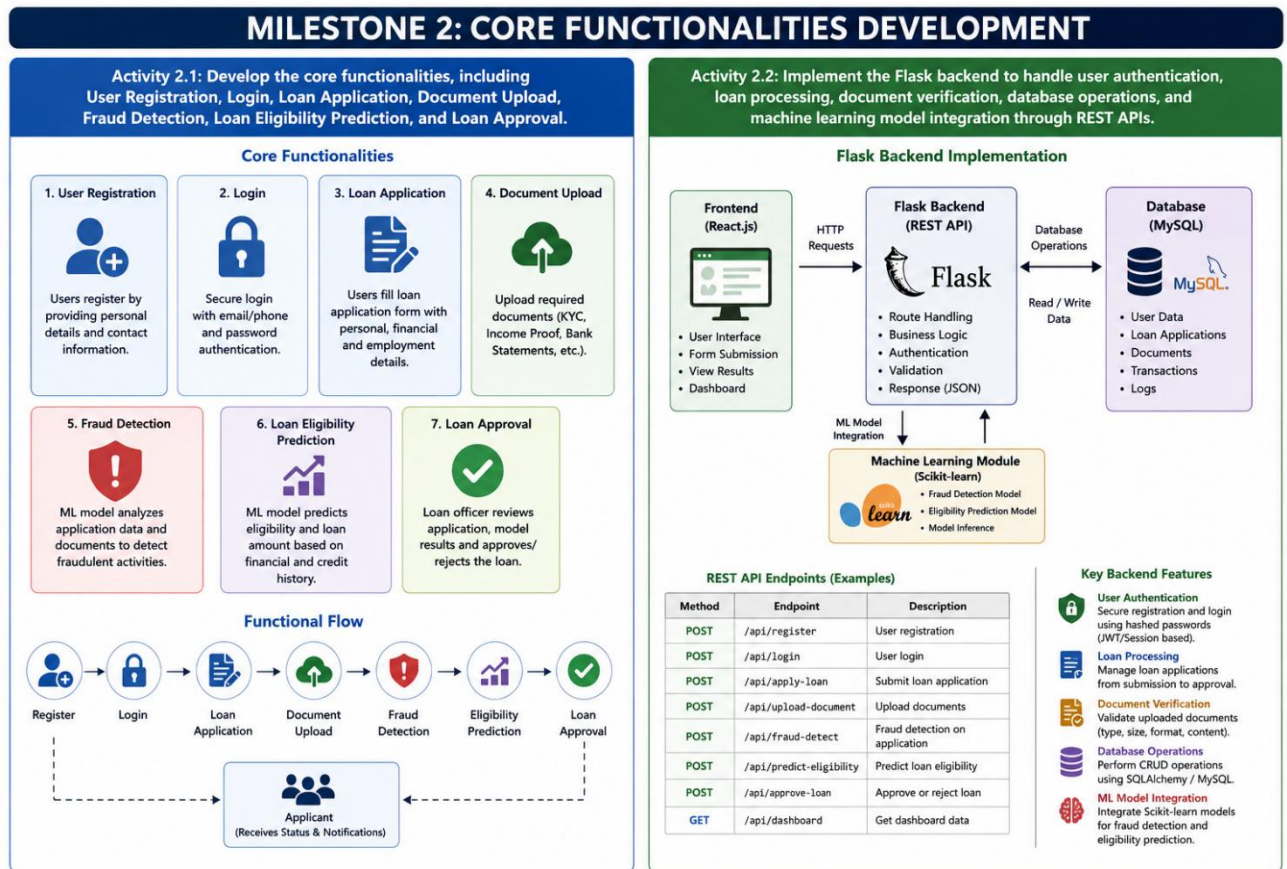
- **Activity 1.1:** Gather project requirements and define the objectives of the Smart Lender system.
- **Activity 1.2:** Research and select the appropriate machine learning algorithm for loan fraud detection and eligibility prediction.
- **Activity 1.3:** Design the application architecture, defining interactions between the frontend, backend, machine learning module, and database.
- **Activity 1.4:** Set up the development environment by installing Python, Flask, React.js, MySQL, Scikit-learn, Pandas, NumPy, and other required dependencies.



### Core Functionalities Development

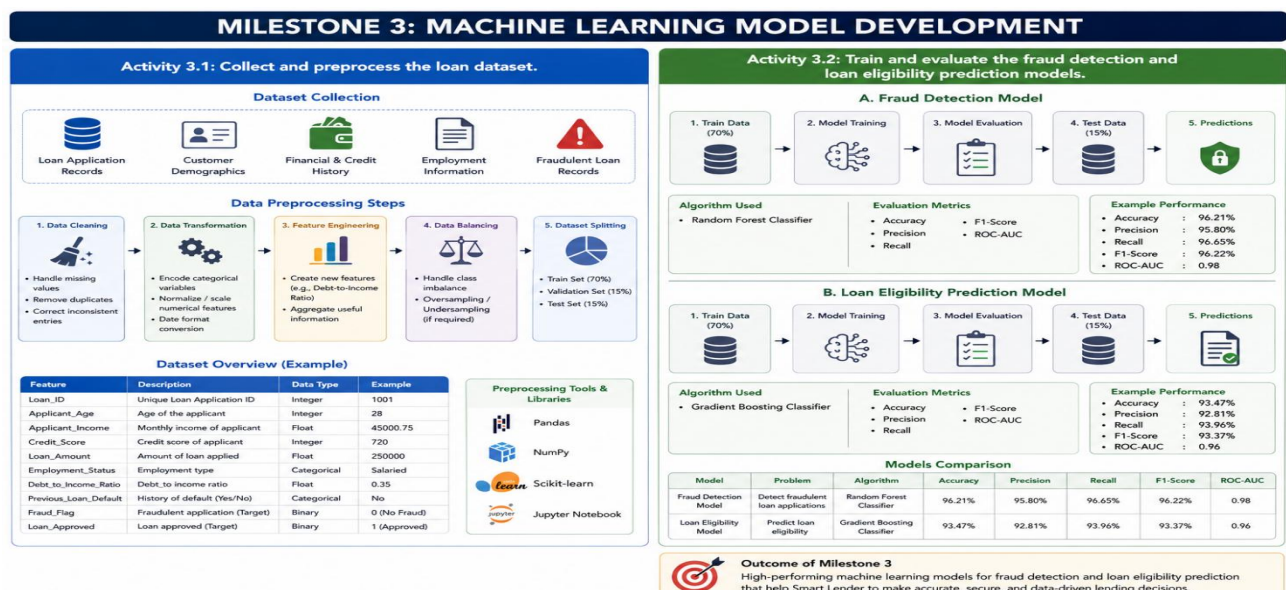
- **Activity 2.1:** Develop the core functionalities, including User Registration, Login, Loan Application, Document Upload, Fraud Detection, Loan Eligibility Prediction, and Loan Approval.
- **Activity 2.2:** Implement the Flask backend to handle user authentication, loan processing, document verification, database operations, and machine learning model

integration through REST APIs.



## Machine Learning Model Development

- **Activity 3.1:** Collect and preprocess the loan dataset.
- **Activity 3.2:** Train and evaluate the fraud detection and loan eligibility prediction models.



## Frontend Development

- **Activity 4.1:** Design and develop responsive user interfaces using React.js, HTML, CSS, and JavaScript.
- **Activity 4.2:** Connect the frontend with backend APIs and display loan status, fraud detection results, and notifications.

**MILESTONE 4: FRONTEND DEVELOPMENT**

**Activity 4.1: Design and develop responsive user interfaces using React.js, HTML, CSS, and JavaScript.**

**Key Features**

  
**User Authentication**  
Login & Registration for users and admins

  
**Loan Application**  
Apply for loan by filling online form

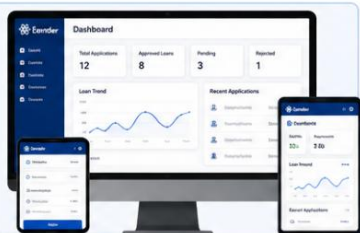
  
**Document Upload**  
Upload KYC and financial documents

  
**Dashboard**  
Track loan status and predictions

  
**Notifications**  
Receive alerts and application updates

**UI/UX Highlights**

- ✓ Responsive Design for Desktop, Tablet & Mobile
- ✓ Clean and Modern UI with Intuitive Navigation
- ✓ Reusable React Components
- ✓ Form Validation and Error Handling
- ✓ Interactive Dashboards and Data Visualization

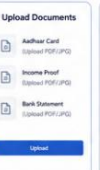


**Pages / Screens Developed**

  
1. Login Page

  
2. Register Page

  
3. Loan Application

  
4. Document Upload

  
5. Dashboard

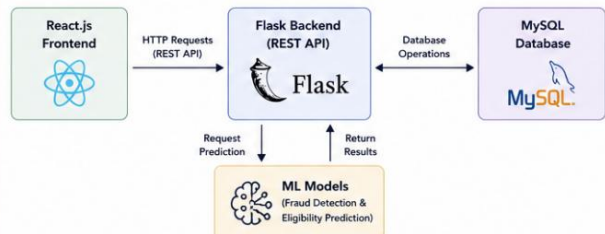
  
6. Notifications

**Tech Stack (Frontend)**

 React.js
  HTML5
  CSS3
  JavaScript
  Bootstrap / Tailwind

**Activity 4.2: Connect the frontend with backend APIs and display loan status, fraud detection results, and notifications.**

**Frontend – Backend Integration Flow**



**API Integration Examples**

| Feature                | API Endpoint           | Method | Purpose                            |
|------------------------|------------------------|--------|------------------------------------|
| User Login             | /api/auth/login        | POST   | Authenticate user and return token |
| Loan Application       | /api/loans/apply       | POST   | Submit loan application            |
| Document Upload        | /api/documents/upload  | POST   | Upload and store documents         |
| Fraud Detection        | /api/ml/fraud-detect   | POST   | Get fraud detection result         |
| Eligibility Prediction | /api/ml/eligibility    | POST   | Get eligibility score and result   |
| Loan Status            | /api/loans/status/{id} | GET    | Get loan application status        |
| Notifications          | /api/notifications     | GET    | Fetch user notifications           |

**Data Display on Frontend**

**Loan Status**

 Status: **Approved**

 Amount: ₹ 250,000

 Updated on: 10 May 2024

**Fraud Detection Result**

 Risk Level: **Low Risk**

 Score: 0.18 / 1.00

 Model: Random Forest

**Eligibility Prediction**

 Eligibility Score: **82%**

 Result: **Eligible**

 Confidence: **High**

**Notifications**

 Loan approved. Congratulations! 1 min ago

 EMI schedule generated. 1 hour ago

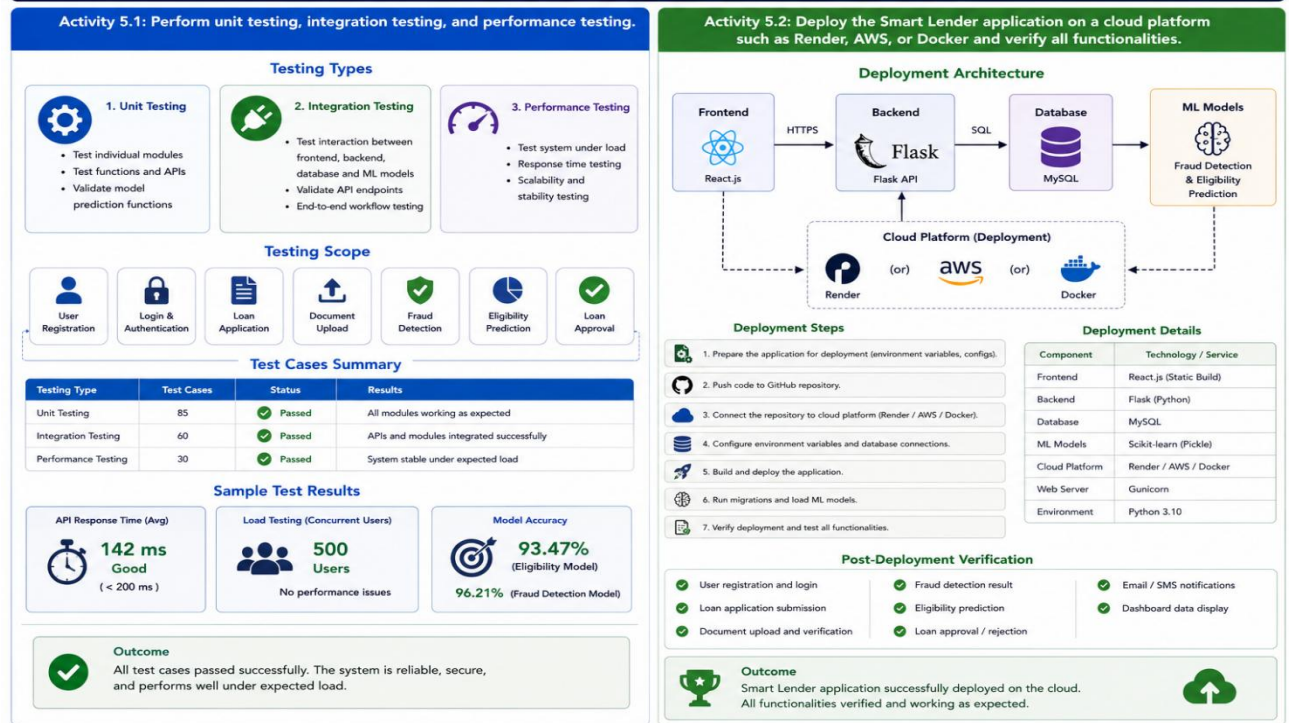
 Disbursal in progress. 2 hours ago

✓ **Outcome:** Frontend is fully connected with backend APIs and provides real-time updates on loan status, fraud detection, eligibility results, and notifications.

## Testing and Deployment

- **Activity 5.1:** Perform unit testing, integration testing, and performance testing.
- **Activity 5.2:** Deploy the Smart Lender application on a cloud platform such as Render, AWS, or Docker and verify all functionalities.

## MILESTONE 5: TESTING AND DEPLOYMENT



## Milestone 2:

### Data Preparation and Model Development

- Activity 1.1: Dataset Collection
- Activity 1.2: Data Preprocessing
- Activity 1.3: Model Training
- Activity 1.4: Model Saving (model.pkl, scaler.pkl)

### Flask Application Development

- Activity 2.1: Develop the Flask Application (app.py)
- Activity 2.2: Integrate the Trained Model with Flask

### Frontend Development

- Activity 3.1: Design the Home Page (home.html)
- Activity 3.2: Develop the Input Form (input.html)
- Activity 3.3: Display the Prediction Result (output.html)

## Testing and Deployment

- Activity 4.1: Test the Loan Prediction System
- Activity 4.2: Run the Flask Application
- Activity 4.3: Verify the Prediction Results

### Conclusion:

Review the project outcomes, evaluate the fraud detection accuracy and loan processing performance, document the results, and identify future enhancements such as real-time credit bureau integration and advanced AI models

## Milestone 3: Data Preparation and Model Development

### Activity 1.1: Dataset Collection

#### Objective

To collect the loan approval dataset required for training the machine learning model.

#### Description:

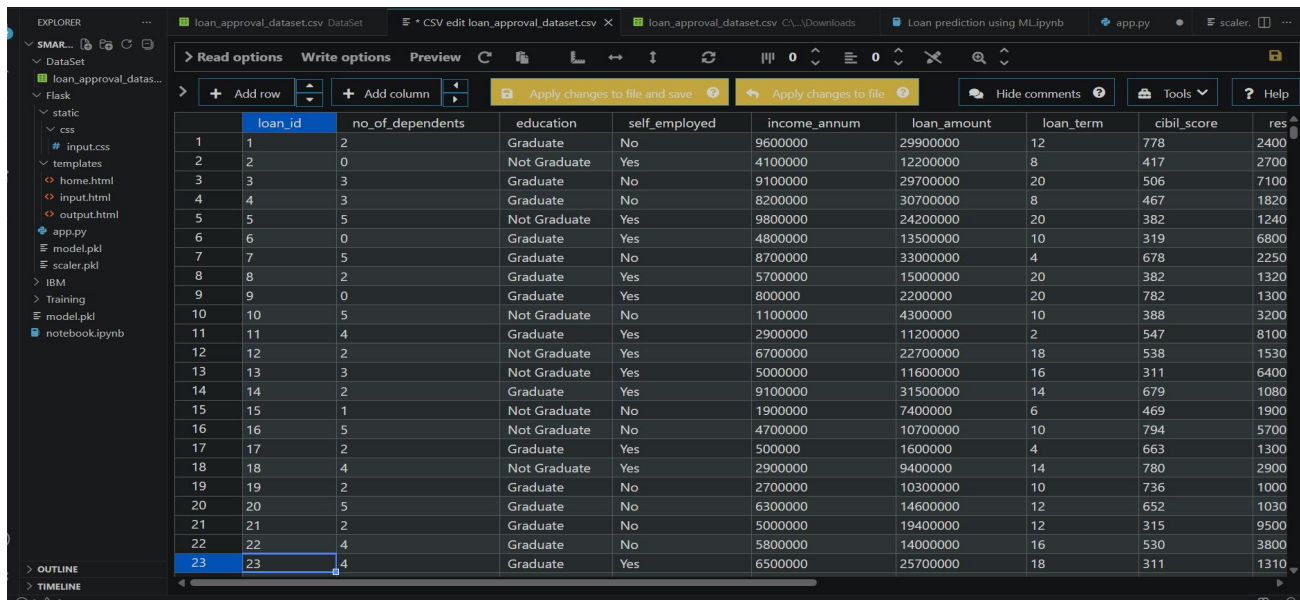
The loan approval dataset was loaded into the Jupyter Notebook using the Pandas library. The column names were cleaned by removing unnecessary spaces, and the first five records were displayed using `df.head()` to verify that the dataset was loaded successfully.

#### Tools Used

- Python, Pandas, Jupyter notebook, Scikit-learn

**Dataset File:** loan\_approval\_dataset.csv

```
# Step 2: Load Dataset
df = pd.read_csv("DataSet/loan_approval_dataset.csv")
df.columns = df.columns.str.strip()
print(df.head())
```



|    | loan_id | no_of_dependents | education    | self_employed | income_annum | loan_amount | loan_term | cibil_score | res  |
|----|---------|------------------|--------------|---------------|--------------|-------------|-----------|-------------|------|
| 1  | 1       | 2                | Graduate     | No            | 9600000      | 29900000    | 12        | 778         | 2400 |
| 2  | 2       | 0                | Not Graduate | Yes           | 4100000      | 12200000    | 8         | 417         | 2700 |
| 3  | 3       | 3                | Graduate     | No            | 9100000      | 29700000    | 20        | 506         | 7100 |
| 4  | 4       | 3                | Graduate     | No            | 8200000      | 30700000    | 8         | 467         | 1820 |
| 5  | 5       | 5                | Not Graduate | Yes           | 9800000      | 24200000    | 20        | 382         | 1240 |
| 6  | 6       | 0                | Graduate     | Yes           | 4800000      | 13500000    | 10        | 319         | 6800 |
| 7  | 7       | 5                | Graduate     | No            | 8700000      | 33000000    | 4         | 678         | 2250 |
| 8  | 8       | 2                | Graduate     | Yes           | 5700000      | 15000000    | 20        | 382         | 1320 |
| 9  | 9       | 0                | Graduate     | Yes           | 800000       | 2200000     | 20        | 782         | 1300 |
| 10 | 10      | 5                | Not Graduate | No            | 1100000      | 4300000     | 10        | 388         | 3200 |
| 11 | 11      | 4                | Graduate     | Yes           | 2900000      | 11200000    | 2         | 547         | 8100 |
| 12 | 12      | 2                | Not Graduate | Yes           | 6700000      | 22700000    | 18        | 538         | 1530 |
| 13 | 13      | 3                | Not Graduate | Yes           | 5000000      | 11600000    | 16        | 311         | 6400 |
| 14 | 14      | 2                | Graduate     | Yes           | 9100000      | 31500000    | 14        | 679         | 1080 |
| 15 | 15      | 1                | Not Graduate | No            | 1900000      | 7400000     | 6         | 469         | 1900 |
| 16 | 16      | 5                | Not Graduate | No            | 4700000      | 10700000    | 10        | 794         | 5700 |
| 17 | 17      | 2                | Graduate     | Yes           | 500000       | 1600000     | 4         | 663         | 1300 |
| 18 | 18      | 4                | Not Graduate | Yes           | 2900000      | 9400000     | 14        | 780         | 2900 |
| 19 | 19      | 2                | Graduate     | No            | 2700000      | 10300000    | 10        | 736         | 1000 |
| 20 | 20      | 5                | Graduate     | No            | 6300000      | 14600000    | 12        | 652         | 1030 |
| 21 | 21      | 2                | Graduate     | No            | 5000000      | 19400000    | 12        | 315         | 9500 |
| 22 | 22      | 4                | Graduate     | No            | 5800000      | 14000000    | 16        | 530         | 3800 |
| 23 | 23      | 4                | Graduate     | Yes           | 6500000      | 25700000    | 18        | 311         | 1310 |

## Activity 1.2: Data Preprocessing

### Description

The dataset was preprocessed before training the machine learning model. Extra spaces in text columns were removed using string operations. Categorical features such as education and self\_employed were converted into numerical values using LabelEncoder. The target column loan\_status was also converted into binary values (Approved = 1, Rejected = 0) to make the data suitable for model training.

```
# Step 4: Remove spaces from text columns
for Loading... select_dtypes(include="object"):
    df[col] = df[col].str.strip()

# Step 5: Encode Categorical Data
encoder = LabelEncoder()

df["education"] = encoder.fit_transform(df["education"])
df["self_employed"] = encoder.fit_transform(df["self_employed"])

df["loan_status"] = df["loan_status"].map({
    "Approved": 1,
    "Rejected": 0
})
```

**Result:** The dataset was successfully cleaned and converted into a machine-learning-ready format by removing unwanted spaces and encoding categorical variables.

## Activity 1.3: Model Training

### Description

The preprocessed dataset was divided into training and testing sets using the train\_test\_split() function. The features were standardized using StandardScaler to improve model performance. A Random Forest Classifier was then trained using the training dataset to predict loan approval status.

```
# Step 7: Features and Target
X = df.drop(["loan_id", "loan_status"], axis=1)
y = df["loan_status"]

# Step 8: Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# Step 9: Feature Scaling
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Step 10: Train Model
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

model.fit(X_train, y_train)
```

## Result

The Random Forest Classifier model was successfully trained using the processed dataset and is ready to predict loan approval outcomes.

## Activity 1.4: Model Evaluation and Model Saving

### Description

After training the model, predictions were generated using the test dataset. The model performance was evaluated by calculating the accuracy score, generating the classification report, and displaying the confusion matrix. Finally, the trained model was saved as **model.pkl** for future predictions.

```
# Step 11: Prediction
predictions = model.predict(x_test)

# Step 12: Accuracy
accuracy = accuracy_score(y_test, predictions)

print("\nAccuracy:", round(accuracy * 100, 2), "%")

# Step 13: Classification Report
print("\nClassification Report")
print(classification_report(y_test, predictions))
```

```
# Step 14: Confusion Matrix
cm = confusion_matrix(y_test, predictions)

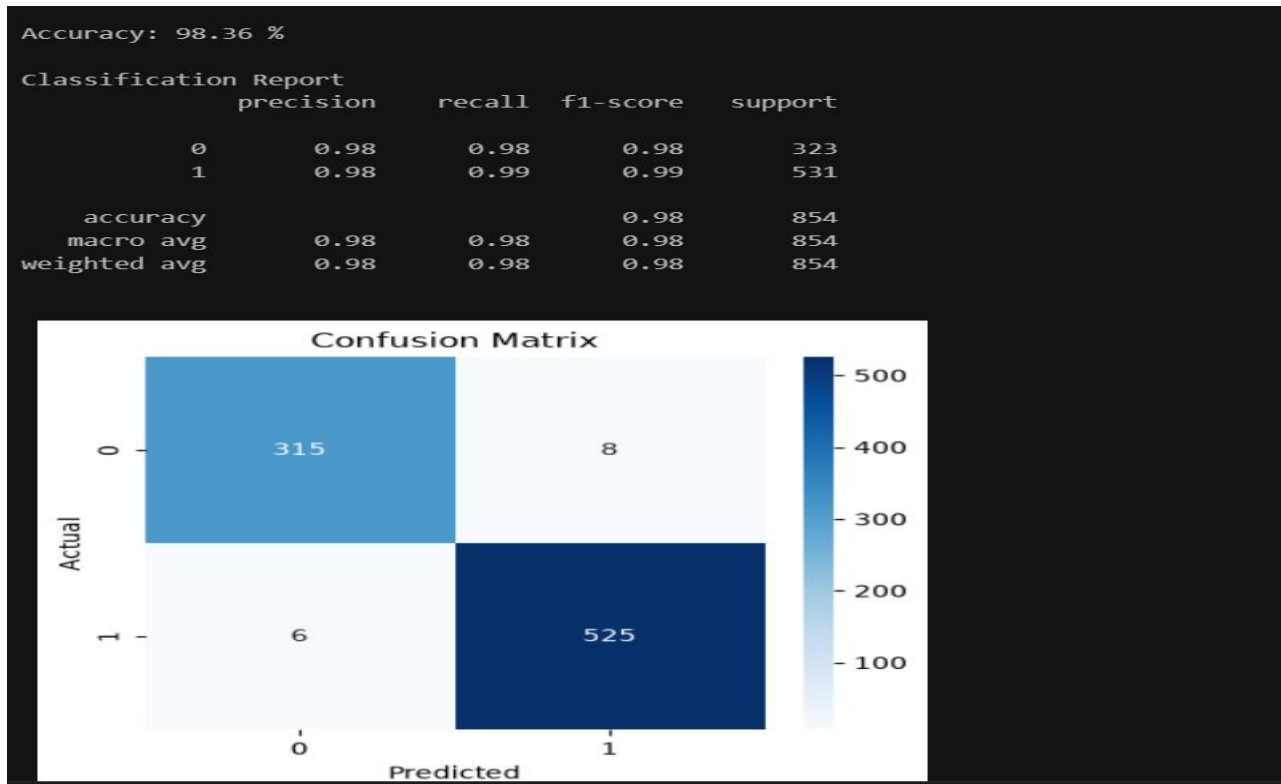
plt.figure(figsize=(5,4))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues"
)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()

# Step 15: Save Model
joblib.dump(model, "model.pkl")

print("\nModel saved successfully as model.pkl")
```

### Result:

The Random Forest model achieved good prediction accuracy. The trained model was successfully saved as model.pkl, which can be used later by the Flask application to predict loan approval.



## Milestone 4: Flask Application Development

### Activity 2.1: Flask Project Structure

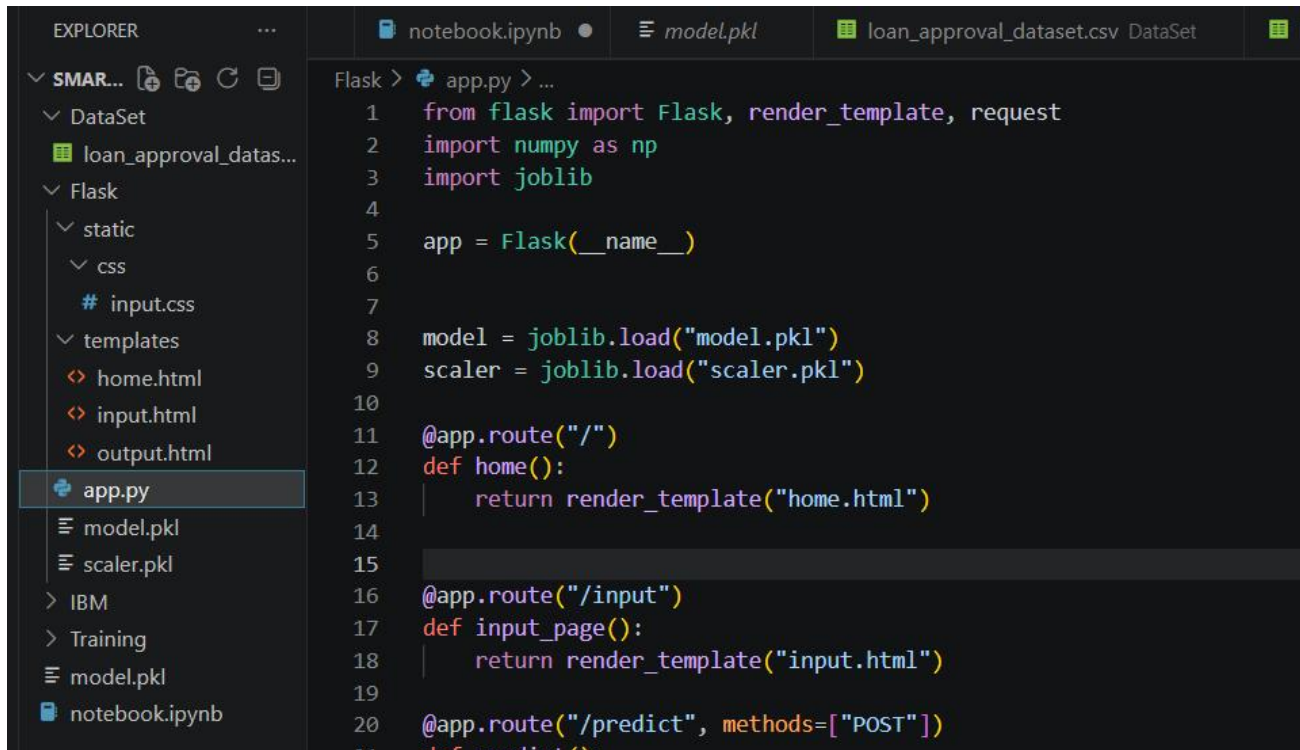
#### Objective:

To organize the project files and folders required for developing the Smart Lender web application using the Flask framework.

#### Description:

The Smart Lender project was organized using the Flask framework. The project structure includes the templates folder for HTML pages, the static folder for CSS files, the trained machine learning model (model.pkl), the scaler (scaler.pkl), and the main application file (app.py). This structure helps maintain a well-organized and scalable web application.

**Figure:** Flask Project Structure of the Smart Lender Application



```

Flask > app.py > ...
1  from flask import Flask, render_template, request
2  import numpy as np
3  import joblib
4
5  app = Flask(__name__)
6
7
8  model = joblib.load("model.pkl")
9  scaler = joblib.load("scaler.pkl")
10
11 @app.route("/")
12 def home():
13     return render_template("home.html")
14
15
16 @app.route("/input")
17 def input_page():
18     return render_template("input.html")
19
20 @app.route("/predict", methods=["POST"])
21 def predict():
  
```

## Result:

The Flask project structure was successfully created and organized for developing the Smart Lender application.

## Activity 2.2: Application Development using app.py

### Objective

To develop the main Flask application that connects the trained machine learning model with the web interface.

### Description

The app.py file is the main component of the Smart Lender application. It initializes the Flask server, loads the trained machine learning model (model.pkl) and scaler (scaler.pkl), receives user input from the HTML form, processes the input data, predicts the loan approval status, and displays the prediction result on the output page.

### Tools Used

- Python
- Flask
- Joblib

- Scikit-learn

**Figure:** Main Flask Application(app.py)

```
Flask > app.py > ...
1  from flask import Flask, render_template, request
2  import numpy as np
3  import joblib
4
5  app = Flask(__name__)
6
7
8  model = joblib.load("model.pkl")
9  scaler = joblib.load("scaler.pkl")
10
11 @app.route("/")
12 def home():
13     return render_template("home.html")
14
15
16 @app.route("/input")
17 def input_page():
18     return render_template("input.html")
19
20 @app.route("/predict", methods=["POST"])
21 def predict():
22
```

## Result

The Flask application successfully loads the trained machine learning model, accepts user input, performs loan approval prediction, and displays the result through the web interface.

## Milestone 3: Frontend Development

### Activity 3.1: HTML Templates Development

#### Objective

To design a user-friendly web interface for the Smart Lender application using HTML templates.

#### Description

The frontend of the Smart Lender application was developed using three HTML templates:

- **home.html** – Displays the welcome page and allows users to start the loan prediction process.

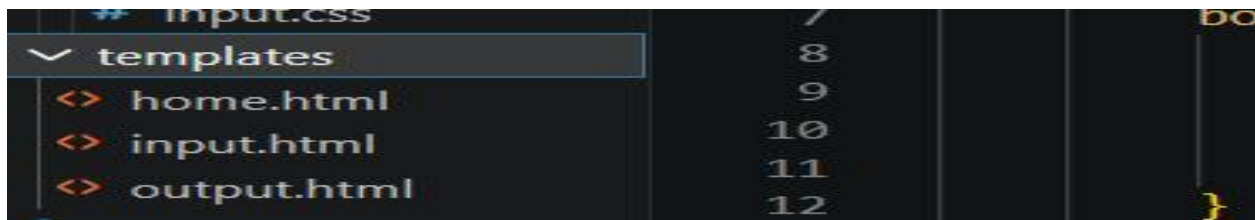
- **input.html** – Provides a form where users enter applicant details such as income, loan amount, education, and CIBIL score.
- **output.html** – Displays the final loan approval prediction result (Approved or Rejected).

These templates are connected to the Flask backend and provide a smooth user experience.

### Tools Used

- HTML5
- CSS3
- Flask Templates

**Figure:** HTML Templates Used in the Smart Lender Application



### Result:

The HTML templates were successfully created and integrated with the Flask application, providing an interactive interface for loan approval prediction.

## Application Execution

### Activity 3.2: Running the Flask Application

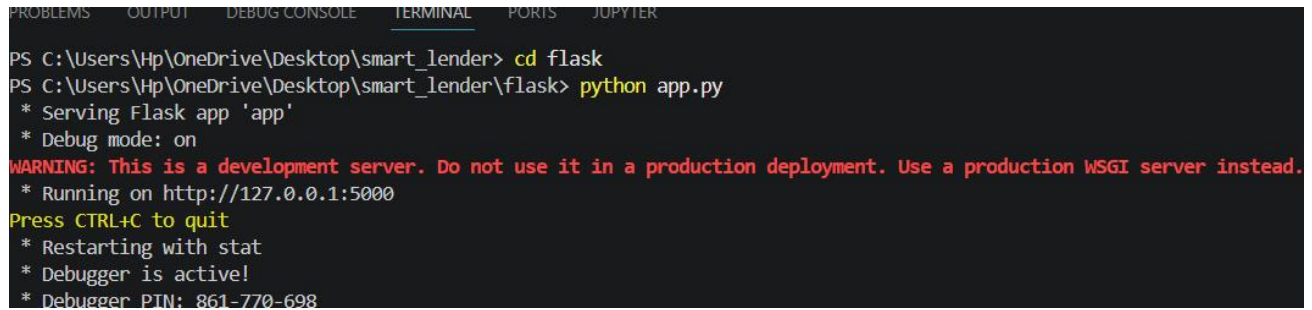
#### Objective

To execute the Flask application and verify that the Smart Lender loan prediction system runs successfully on the local server.

#### Description

The Flask application was executed using the terminal in Visual Studio Code. The app.py file was run using the Python interpreter. After the server started successfully, the application became accessible through the local host URL in the web browser.

**Figure:** Running the Flask Application using VS Code Terminal



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER
PS C:\Users\Hp\OneDrive\Desktop\smart_lender> cd flask
PS C:\Users\Hp\OneDrive\Desktop\smart_lender\flask> python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 861-770-698
```

## Result

The Flask application started successfully, and the Smart Lender web application became available through the local server for loan prediction.

## User Interface

### Activity 3.3: Home Page

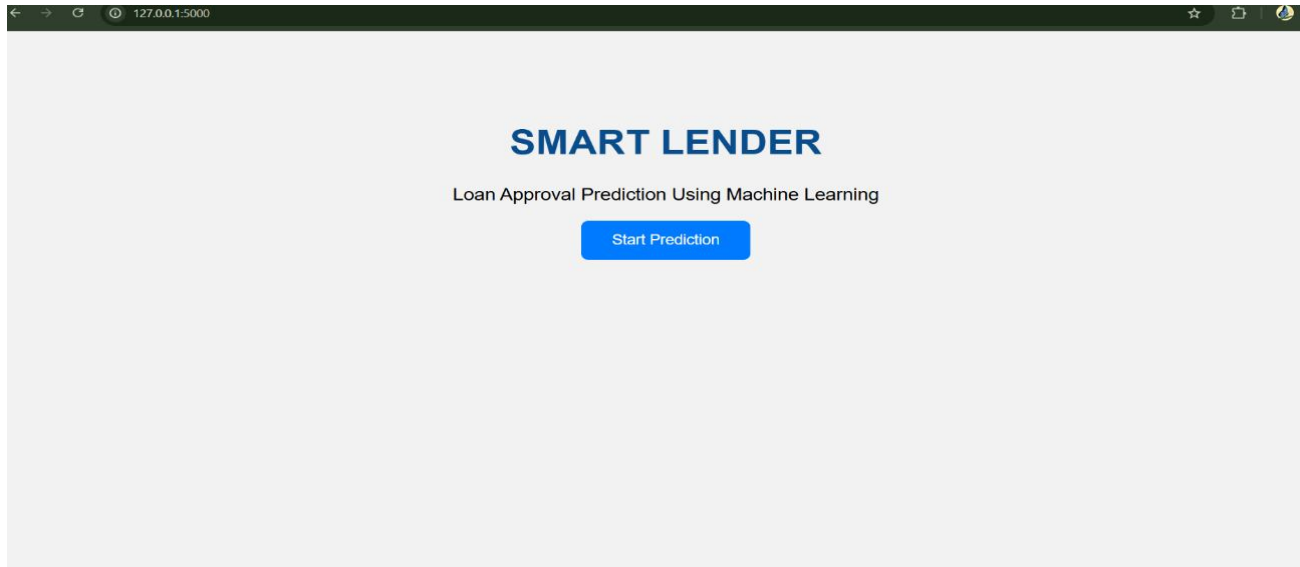
#### Objective

To provide a welcome page for the Smart Lender application and allow users to start the loan prediction process.

#### Description

The Home Page serves as the entry point of the Smart Lender application. It displays the project title and provides a button to navigate to the loan prediction form. The page is designed to provide a simple and user-friendly interface.

**Figure:** Smart Lender Home Page



## Result

The Home Page was successfully developed and provides users with easy access to the loan prediction application.

### Activity 3.4: Loan Details Input Page

#### Objective

To provide a form for users to enter the required loan applicant details for loan approval prediction.

#### Description

The Input Page allows users to enter loan applicant information such as education, self-employment status, annual income, loan amount, loan term, CIBIL score, residential assets, commercial assets, luxury assets, bank assets, and the number of dependents. After entering the required details, users can submit the form to predict the loan approval status.

**Figure: Loan Details Input Form**

## Loan Prediction Form

**No of Dependents**

**Education**

**Self Employed**

**Annual Income**

**Loan Amount**

**Loan Term**

**CIBIL Score**

**Residential Assets Value**

### Result

The input form was successfully developed and allows users to enter loan details for prediction.

# Loan Prediction Result

## Activity 3.5: Loan Prediction Output

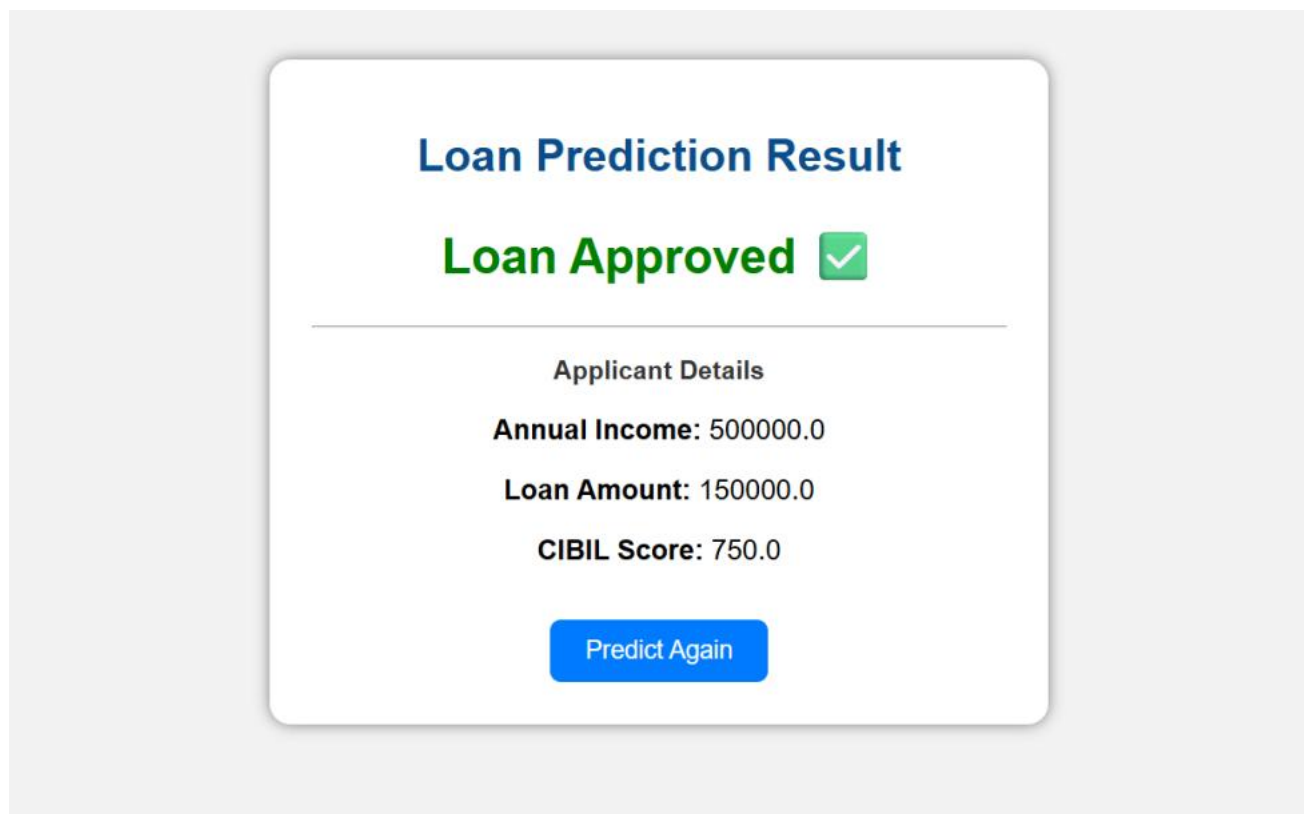
### Objective

To display the loan approval prediction result based on the details entered by the user.

### Description

After the user submits the loan details, the trained Machine Learning model processes the input data and predicts whether the loan application is **Approved** or **Rejected**. The prediction result is displayed on the output page.

**Figure:** Loan Prediction Result



### Result

The application successfully predicts the loan approval status and displays the result to the user.

## **Future Enhancements**

- Improve prediction accuracy using advanced Machine Learning algorithms.
- Add user authentication and login functionality.
- Store prediction history in a database.
- Deploy the application on a cloud platform for public access.
- Enhance the user interface with modern design and responsive layouts.

## **Conclusion**

The Smart Lender – Loan Approval Prediction Using Machine Learning project was successfully developed and implemented. The system uses a Random Forest Classifier to predict whether a loan application will be approved or rejected based on the applicant's details. The dataset was preprocessed, the model was trained and evaluated, and the trained model was integrated with a Flask web application. The application provides a simple and user-friendly interface for entering loan details and viewing prediction results. This project demonstrates the practical use of Machine Learning and Web Development in building an intelligent loan approval prediction system.